Министерство цифрового развития, связи и массовых коммуникаций РФ
ФГБОУ «Сибирский государственный университет телекоммуникаций и информатики (СибГУТИ)
Уральский технический институт связи и информатики (филиал)
в г. Екатеринбурге (УрТИСИ СибГУТИ)

Кафедра информационных систем и технологий

Учебная практика

МДК 02.02 Инструментальные средства разработки программного обеспечения

Пояснительная записка

Студент Шестаков И.В.

Группа 284

Екатеринбург - 2026

Министерство цифрового развития, связи и массовых коммуникаций РФ
ФГБОУ «Сибирский государственный университет телекоммуникаций и информатики (СибГУТИ)
Уральский технический институт связи и информатики (филиал)
в г. Екатеринбурге (УрТИСИ СибГУТИ)

ОТЗЫВ

о работе обучающегося Шестакова Игоря Владимировича

в период подготовки учебного проекта по теме «Разработка веб-приложения для магазина по продаже мебели»

Направление подготовки: 09.02.07 Информационные системы и программирование

Преподаватель кафедры ИСТ УрТИСИ

Преподаватель _____ Бурумбаев Д. И.

(Бурумбаев Даниль Ильмирович)

«_____» _____

Содержание

Введение 4
1 Анализ предметной области 5
2 Проектирование базы данных 6
2.1 Описание базы данных 6
2.2 ER-диаграмма 7
3 Разработка серверной части 8
3.1 Описание API 8
4 Пользовательский интерфейс 10
4.1 Описание пользовательского интерфейса 10
4.2 Страница со списком товаров 10
4.3 Страница со списком цехов 12
4.4 Страница добавления и редактирования товаров 13
5 Описание дипломного проекта 17
5.1 Обоснование выбора инструментальных средств разработки 17
5.2 Структура приложения 17
5.3 Инструкция по работе с приложением 17
Заключение 20
Приложение А 21

Введение

Разработка веб-приложения для управления ассортиментом мебельной компании позволит автоматизировать процессы учёта и редактирования информации о продукции, которые ранее выполнялись вручную или с использованием неспециализированных инструментов. Это сократит время сотрудников на внесение изменений в каталог, минимизирует вероятность ошибок при вводе данных и обеспечит актуальность информации о товарах. Внедрение системы также улучшит взаимодействие между отделами компании, так как все данные о продукции будут храниться централизованно и быть доступны уполномоченным сотрудникам.

Актуальность темы обусловлена необходимостью цифровизации малого и среднего бизнеса, в том числе в сфере производства и продажи мебели. Существующие решения часто либо избыточны по функционалу и стоимости, либо не учитывают специфику конкретного предприятия. Разработка собственного приложения позволяет создать инструмент, точно соответствующий потребностям компании и легко адаптируемый под изменяющиеся бизнес-процессы.

Целью учебной практики является разработка веб-приложения для управления продукцией мебельной компании, обеспечивающего просмотр, добавление и редактирование данных о товарах.

Задачи:

- разработать пользовательский веб-интерфейс для просмотра и редактирования каталога продукции;
- спроектировать и создать базу данных для хранения информации о товарах, типах продукции, материалах и производственных цехах;
- реализовать серверную часть приложения для взаимодействия с базой данных;
- разработать функционал расчёта времени изготовления продукции и количества необходимого сырья;
- обеспечить обработку ошибок и информирование пользователя о результатах операций.

1 Анализ предметной области

Компания занимается изготовлением мебели. В ассортименте представлены современные и классические модели, для производства используются натуральные и экологичные материалы. Компания расширяет ассортимент изделий, следуя новинкам в стилевом решении, и повышает эффективность производства за счет рационального использования производственного оборудования, поэтому необходима система для отслеживания информации о продукции.

Разработать подсистему для работы с продукцией компании, обеспечивающую следующий функционал:

– просмотр списка продукции;
– добавление/редактирование данных о продукции;
– просмотр списка цехов для производства продукции.

Используемый стек технологий:

- Java 17 – основной язык программирования;
- Spring Boot 4 – фреймворк для создания веб-приложения;
- Spring Data JPA (Hibernate) – ORM для работы с базой данных;
- PostgreSQL – СУБД;
- Thymeleaf – шаблонизатор для генерации HTML-страниц;
- HTML/CSS – вёрстка пользовательского интерфейса;
- Maven – система сборки и управления зависимостями;
- pgAdmin – инструмент для администрирования базы данных.

2 Проектирование базы данных

2.1 Описание базы данных

В качестве СУБД использовалась PostgreSQL. Для взаимодействия с базой данных применяется Spring Data JPA (Hibernate) – ORM, позволяющая работать с таблицами как с Java-объектами и автоматически генерировать SQL-запросы. Создание базы данных выполнялось с помощью SQL-скрипта через pgAdmin, заполнение данными осуществлялось из файлов, предоставленных заказчиком.

Таблицы базы данных

1. Таблица «Типы продукции» (product_types):
- id (INTEGER, PRIMARY KEY) – уникальный идентификатор типа;
- name (VARCHAR) – название типа продукции (например, «Гостиные», «Кровати»);
- coefficient (NUMERIC) – коэффициент типа продукции (используется в расчётах).

2. Таблица «Типы материалов» (material_types):
- id (INTEGER, PRIMARY KEY);
- name (VARCHAR) – название материала (например, «Мебельный щит из массива дерева», «ЛДСП»);
- loss_percent (NUMERIC) – процент потерь сырья.

3. Таблица «Цехи» (workshops):
- id (INTEGER, PRIMARY KEY);
- name (VARCHAR) – название цеха;
- type (VARCHAR) – тип цеха;
- workers_count (INTEGER) – количество человек в цехе.

4. Таблица «Продукция» (products)
- id (INTEGER, PRIMARY KEY);
- name (VARCHAR) – наименование продукции;
- article (VARCHAR) – артикул товара;
- price (NUMERIC) – минимальная стоимость для партнёра (с двумя знаками после запятой);
- type_id (INTEGER, FOREIGN KEY REFERENCES product_types(id)) – идентификатор типа продукции;
- material_id (INTEGER, FOREIGN KEY REFERENCES material_types(id)) – идентификатор основного материала.

Таблица «Продукция и цехи» (product_workshops) – связь многие-ко-многим с дополнительным полем:id (INTEGER, PRIMARY KEY)
- id (INTEGER, PRIMARY KEY);
- product_id (INTEGER, FOREIGN KEY REFERENCES products(id)) – продукция;
- workshop_id (INTEGER, FOREIGN KEY REFERENCES workshops(id)) – цех;

– time (NUMERIC) – время изготовления данной продукции в указанном цехе (в часах).

Связи между таблицами:

Продукция связана с типом продукции и материалом через внешние ключи. Каждая запись в таблице product_workshops связывает конкретный продукт с конкретным цехом, указывая время изготовления. Таким образом, для одного продукта может быть несколько цехов, а общее время изготовления вычисляется как сумма значений time по всем цехам, задействованным в производстве этого продукта. Данный расчёт реализован в методе getTotalTime() класса Product.

Заполнение базы данных:

Для первоначального наполнения базы данных использовались SQL-скрипты, выполненные через pgAdmin. Импорт данных осуществлялся из файлов, предоставленных заказчиком (с пометкой "import" в ресурсах). Скрипты создают таблицы в соответствии с третьей нормальной формой и заполняют их тестовыми данными о типах продукции, материалах, цехах, самой продукции и связях между продукцией и цехами. При импорте обеспечивается ссылочная целостность за счёт правильной последовательности заполнения таблиц и использования внешних ключей. Скрипт базы данных находится в директории проекта и при необходимости может быть использован для развёртывания на новом окружении.

2.2 ER-диаграмма

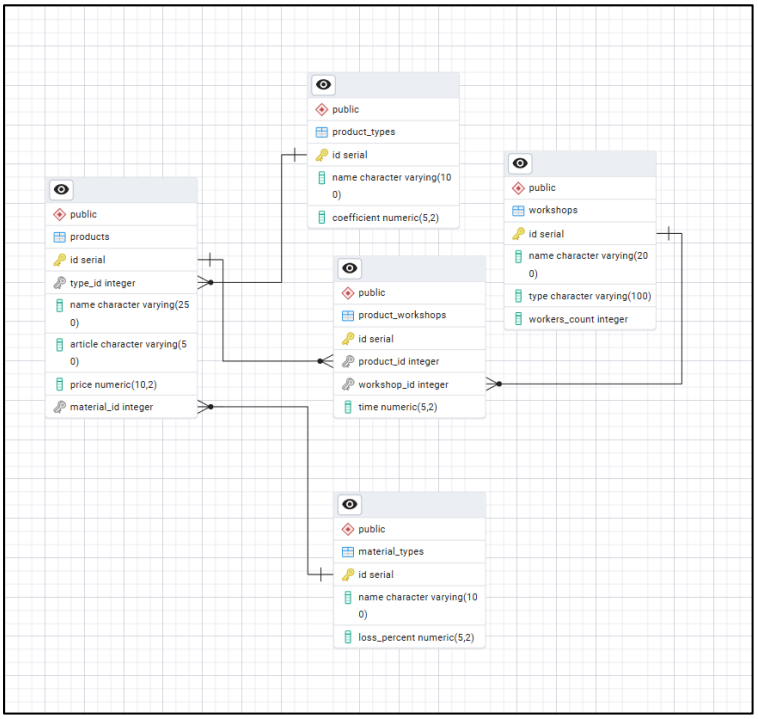

Рисунок 2.1 – ER-диаграмма базы данных

3 Разработка серверной части

3.1 Описание API

Разработанное веб-приложение построено на базе Spring Boot — популярного фреймворка для создания корпоративных приложений на языке Java. В качестве базы данных используется PostgreSQL, взаимодействие с которой осуществляется через Spring Data JPA (Hibernate). Это обеспечивает удобство работы с данными и автоматическое управление подключениями к базе.

Spring Boot обрабатывает HTTP-запросы и формирует ответы, используя шаблонизатор Thymeleaf для генерации HTML-страниц. Spring Data JPA предоставляет объектно-реляционное отображение (ORM) для таблиц базы данных через Entity-классы. Контроллеры обрабатывают входящие запросы, выполняют бизнес-логику и возвращают имя шаблона с передачей данных через модель. Валидация входных данных осуществляется как на стороне клиента (с помощью HTML5-атрибутов), так и на стороне сервера (с помощью проверок в контроллерах).

Реализованные маршруты:

1. Работа с продукцией

- GET / – отображение главной страницы со списком всей продукции. Для каждого продукта выводится тип, наименование, артикул, стоимость, основной материал и рассчитанное общее время изготовления.
- GET /add_product – отображение формы для добавления нового продукта. В форму подгружаются актуальные списки типов продукции и материалов для выпадающих списков.
- GET /update_product/{id} – отображение формы для редактирования существующего продукта. Все поля автоматически заполняются данными из базы.
- POST /save_product – обработка данных формы при добавлении или редактировании продукта. Выполняется валидация (проверка отрицательной цены), установка связанных сущностей и сохранение в базу данных. При успешном сохранении выполняется редирект на главную страницу с сообщением об успехе, при ошибке – возврат на форму с сообщением об ошибке.

2. Работа с цехами

- GET /workshops/{productId} – отображение списка цехов, задействованных в производстве конкретного продукта. Для каждого цеха выводится название, количество человек и время изготовления в данном цехе.

3. Вспомогательные маршруты

- GET /test-calc – тестовый маршрут для проверки метода расчёта сырья. Принимает параметры (тип продукции, тип материала, количество, два

параметра) и возвращает рассчитанное количество сырья с учётом коэффициента типа продукции и процента потерь материала.

Обработка ошибок

В приложении реализована комплексная система обработки ошибок. На стороне контроллеров используется механизм try-catch с пробросом исключений и редиректами через RedirectAttributes, что позволяет передавать информативные сообщения об ошибках на фронтенд. На клиентской стороне сообщения отображаются в виде цветных блоков: зелёные для уведомлений об успешном выполнении операций, красные для сообщений об ошибках. При попытке редактирования несуществующего продукта выполняется редирект на главную страницу с соответствующим сообщением. Валидация на стороне клиента (атрибут min="0" для поля стоимости) предотвращает отправку некорректных данных на сервер.

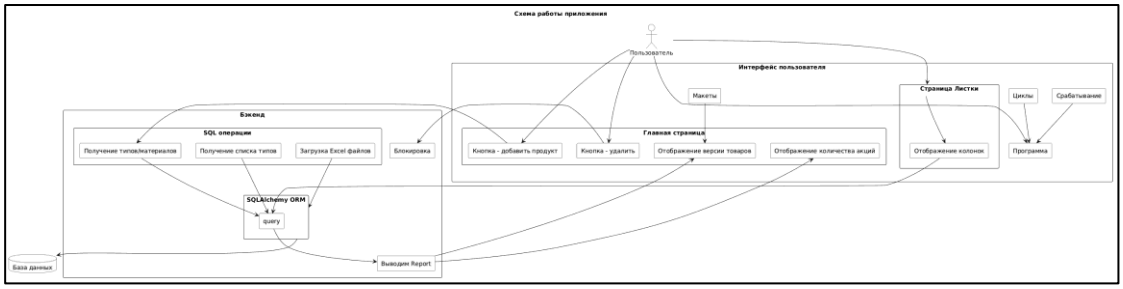

Рисунок 3.1 – Блок-схема приложения

4 Пользовательский интерфейс

4.1 Описание пользовательского интерфейса

Пользовательский интерфейс разработан с использованием шаблонизатора Thymeleaf для генерации HTML-страниц на стороне сервера. Дизайн полностью соответствует руководству по стилю: используется шрифт Candara, основные цвета – белый фон (#FFFFFF), дополнительный фон #D2DFFF и акцентный #355CBD.

На главной странице располагается логотип компании (без искажений), заголовок «Список продуктов» и кнопка добавления нового товара, выделенная акцентным цветом. Все продукты отображаются в виде карточек на фоне #D2DFFF. Каждая карточка содержит информацию о типе, названии, артикуле, стоимости, основном материале, а также рассчитанное время изготовления. Для каждого продукта предусмотрены две кнопки: «Изменить продукт» и «Цеха», выполненные в акцентном цвете.

Форма добавления и редактирования продукта реализована на отдельной странице и содержит поля для ввода артикула, наименования, стоимости, а также выпадающие списки для выбора типа продукции и основного материала. Выпадающие списки динамически заполняются данными из базы при загрузке формы. Кнопки «Сохранить» и «Назад» также оформлены в соответствии с руководством по стилю.

Страница со списком цехов для конкретного продукта отображает информацию в табличном виде: название цеха, количество человек и время изготовления. В верхней части страницы выводится краткая информация о продукте, для которого отображаются цеха. Предусмотрена кнопка возврата к общему списку продукции.

Все страницы имеют соответствующие заголовки и используют общий логотип в качестве иконки. Сообщения об успешном выполнении операций и об ошибках отображаются в виде цветных блоков в верхней части страницы (зелёные для успеха, красные для ошибок), что обеспечивает понятную обратную связь с пользователем.

4.2 Страница со списком товаров

Главная страница приложения доступна по адресу / и отображает список всей продукции в виде карточек на дополнительном фоне. Каждая карточка содержит следующую информацию:

- тип продукции;
- наименование продукции;
- артикул;

– минимальная стоимость для партнёра (в рублях с двумя знаками после запятой);
– основной материал;
– время изготовления в часах (целое число, рассчитываемое как сумма времени по всем цехам).

В нижней части каждой карточки расположены две кнопки, выполненные в акцентном цвете: «Изменить продукт» и «Цеха». Кнопка «Изменить продукт» ведёт на форму редактирования соответствующего продукта (/update_product/{id}), кнопка «Цеха» – на страницу со списком цехов для данного продукта (/workshops/{productId}). В верхней части страницы расположена кнопка «Добавить товар» для перехода к форме создания нового продукта (/add_product).

При выполнении операций добавления или редактирования на главной странице отображаются цветные блоки с сообщениями: зелёный блок для уведомлений об успешном сохранении, красный блок для сообщений об ошибках. Сообщения передаются через механизм RedirectAttributes и отображаются с помощью условных блоков Thymeleaf.

Функционал удаления продуктов не реализован, так как данное требование отсутствовало в задании. Индикатор загрузки не используется, поскольку страница генерируется на сервере и данные доступны сразу при рендеринге. В случае ошибок при загрузке данных (например, при проблемах с подключением к базе) приложение возвращает стандартную страницу ошибки.

Рисунок 4.1 – Страница со списком товаров

4.3 Страница со списком цехов

Страница доступна по адресу /workshops/{productId} и отображает список цехов, задействованных в производстве конкретного продукта. В верхней части

страницы выводится краткая информация о продукте: наименование, артикул и общее время изготовления.

Цеха отображаются в виде таблицы с тремя колонками: название цеха, количество человек и время изготовления в данном цехе. Для каждого цеха выводится:

– название цеха;
– количество человек, работающих в цехе;
– время изготовления продукции в данном цехе (в часах).

Если для продукта не назначено ни одного цеха, отображается соответствующее информационное сообщение. В нижней части страницы расположена кнопка возврата на главную страницу со списком продукции.

Функционал добавления, редактирования или удаления цехов на данной странице не предусмотрен – реализован только просмотр существующих данных.

Рисунок 4.2 – Страница со списком цехов

4.4 Страница добавления и редактирования товаров

Форма открывается по клику на кнопку «Добавить товар» на главной странице или на кнопку «Изменить продукт» на карточке товара. Заголовок формы динамически меняется: «Добавление товара» для нового продукта, «Редактирование товара» для существующего.

Страница содержит форму со следующими полями:

– артикул (текстовое поле, обязательное);
– тип продукта (выпадающий список, заполняется данными из таблицы product_types);
– наименование (текстовое поле, обязательное);
– минимальная стоимость для партнёра (числовое поле с шагом 0.01, обязательное);
– основной материал (выпадающий список, заполняется данными из таблицы material_types).

Внизу формы расположены кнопки «Сохранить» и «Назад» для возврата к списку без сохранения.

Валидация данных выполняется на двух уровнях:

на стороне клиента используется встроенная HTML5-валидация (атрибуты required для обязательных полей, min="0" и step="0.01" для поля стоимости), которая не даёт отправить форму с некорректными данными;

на стороне сервера в контроллере выполняется проверка отрицательной стоимости, при ошибке выбрасывается исключение.

При успешном сохранении выполняется редирект на главную страницу, где отображается зелёный блок с сообщением «Продукт успешно сохранен». При ошибке пользователь возвращается на форму и видит красный блок с описанием проблемы (например, «Ошибка при сохранении: Цена не может быть отрицательной»).

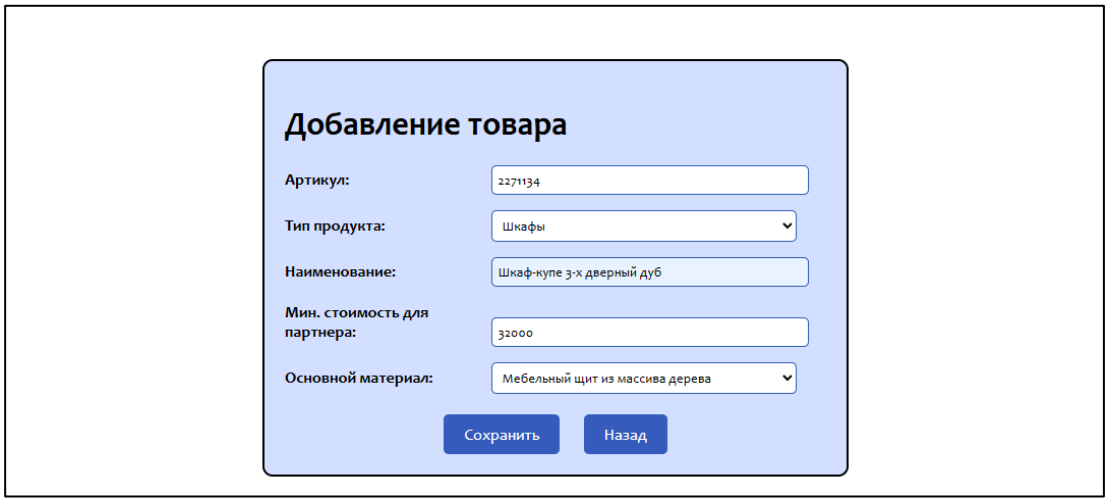

Рисунок 4.3 – Форма добавления товара

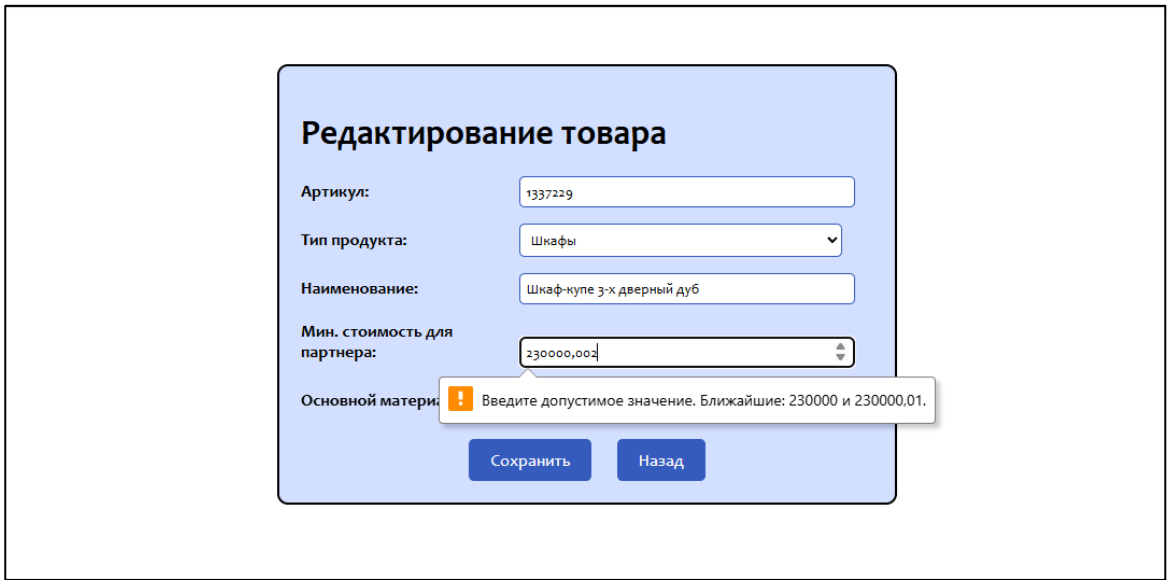

Рисунок 4.4 – Форма редактирования товара

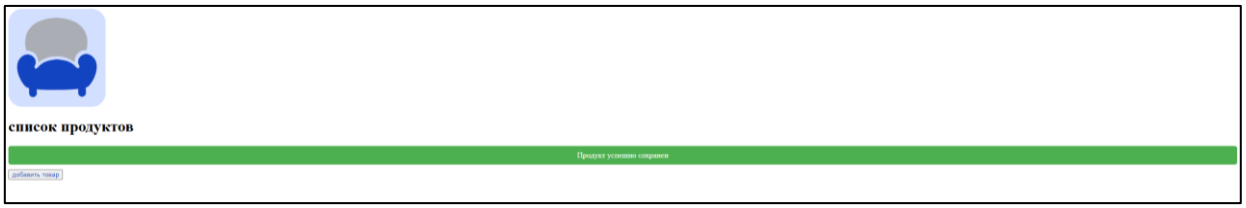

Рисунок 4.5 – Сообщение об успешном сохранении

Выпадающие списки для выбора типа продукции и материала динамически заполняются данными из базы при загрузке формы, что исключает возможность ввода несуществующих значений

Рисунок 4.6 – Сообщение об ошибке

На рисунке 4.6 указано сообщение об ошибке при вводе в поле «Артикул» значения, не поддерживаемого типом данных, который использовался при создании этого поля в базе данных.

5 Описание дипломного проекта

5.1 Обоснование выбора инструментальных средств разработки

Для разработки приложения использовался следующий стек:

- JAVA – основной язык программирования
- Spring – фреймворк для написания серверной части и API;
- HTML/CSS/JS – веб-интерфейс;
- PostgreSQL – СУБД для хранения данных;
- Docker – контейнеризация.

Выбранные инструментальные средства образуют единый и согласованный технологический стек. Java и Spring Boot обеспечивают стабильную серверную часть, PostgreSQL служит надежным средством хранения данных, а докер предоставляет одинаковое окружение для разработки и развертывания. Клиентская часть на HTML, CSS и JavaScript взаимодействует с сервером по принципу клиент-сервер через HTTP-запросы.

Такой набор технологий обеспечивает надежность, удобство сопровождения, стабильность работы и соответствует требованиям учебного проекта

5.2 Структура приложения

Разрабатываемая система представляет собой веб-приложение «Онлайн-журнал посещаемости», предназначенное для фиксации посещаемости студентов преподавателями в режиме реального времени и формирования отчётов для подразделений учебного заведения. Архитектура приложения построена по классической трёхзвенной схеме, включающей клиентскую часть, серверную часть и систему управления базами данных. Клиентская часть обеспечивает отображение данных и взаимодействие с пользователем через браузер. Серверная часть реализована на Java с использованием Spring Boot и выполняет обработку бизнес-логики. Слой хранения реализован в базе данных PostgreSQL. Между клиентом и сервером осуществляется обмен данными в формате JSON через REST API. Сервер отвечает за валидацию входных данных, контроль прав доступа и согласованное выполнение операций с данными.

5.3 Инструкция по работе с приложением

В данном разделе будут указаны основные функции, реализованные в веб-приложении по указанным требованиям, подробное описание всех страниц и работа с ними.

Для работы с системой преподаватель или администратор должны выполнить вход с использованием логина и пароля. Система различает роли пользователей: преподаватель имеет доступ к функциям отметки посещаемости и просмотра статистики, администратор - к управлению пользователями и справочниками.

После успешного входа преподаватель попадает на главный экран, где отображается навигационная панель и панель фильтров. Для отметки посещаемости необходимо выбрать конкретную дату. После выбора даты система формирует таблицу студентов, для которых преподаватель может установить статус посещаемости: присутствовал, отсутствовал, опоздал или освобождён.

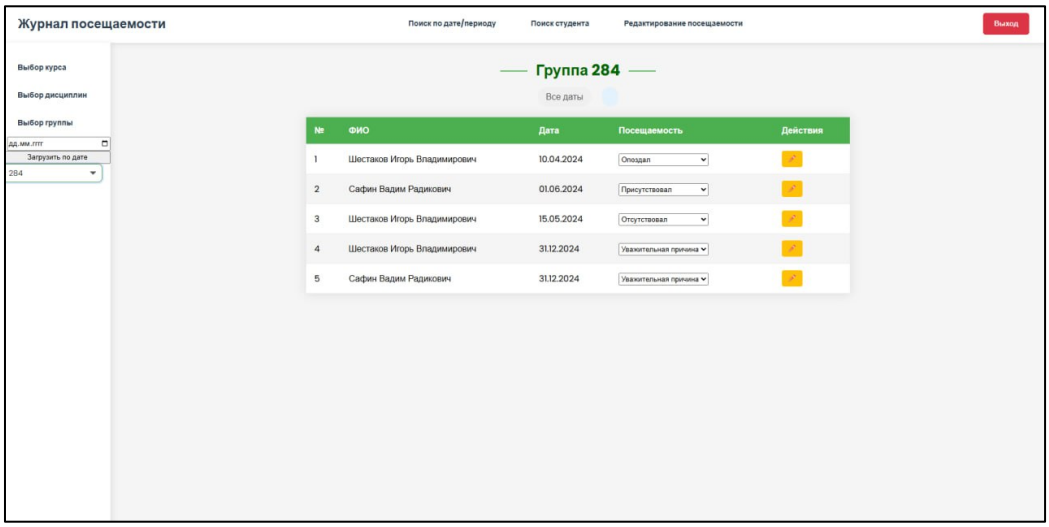

Рисунок 5.1 - Главный экран программы с выбором даты

Преподаватель также может выбрать конкретного студента для просмотра его посещаемости в текущем учебном году. На экране отображается история посещаемости студента с возможностью просмотреть статус на каждое занятие и при необходимости изменить его.

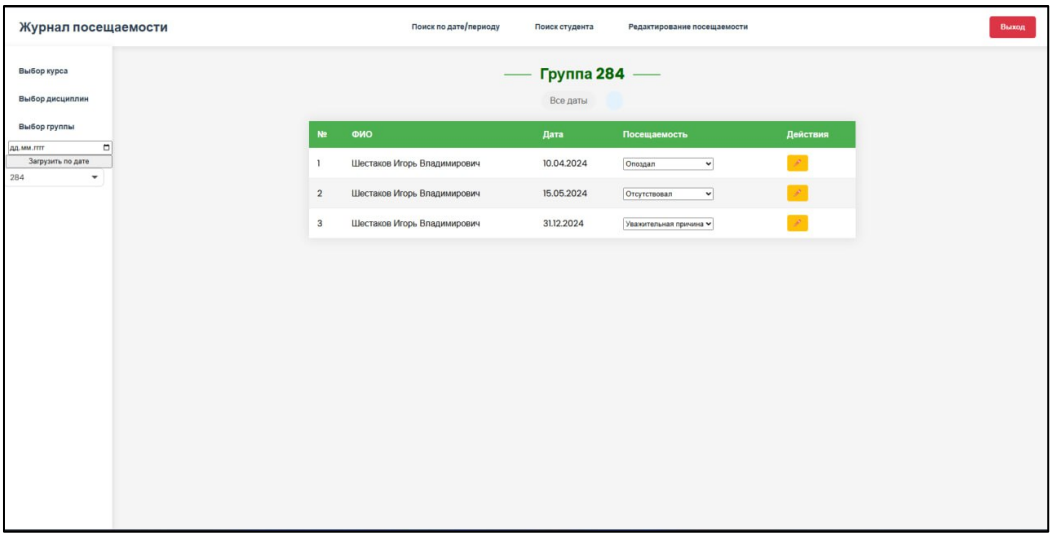

Рисунок 5.2 - Просмотр посещаемости выбранного студента

Для работы с группами реализована возможность выбора группы через фильтр. После выбора группы система отображает список студентов выбранной группы и позволяет массово или поштучно отмечать посещаемость на выбранную дату.

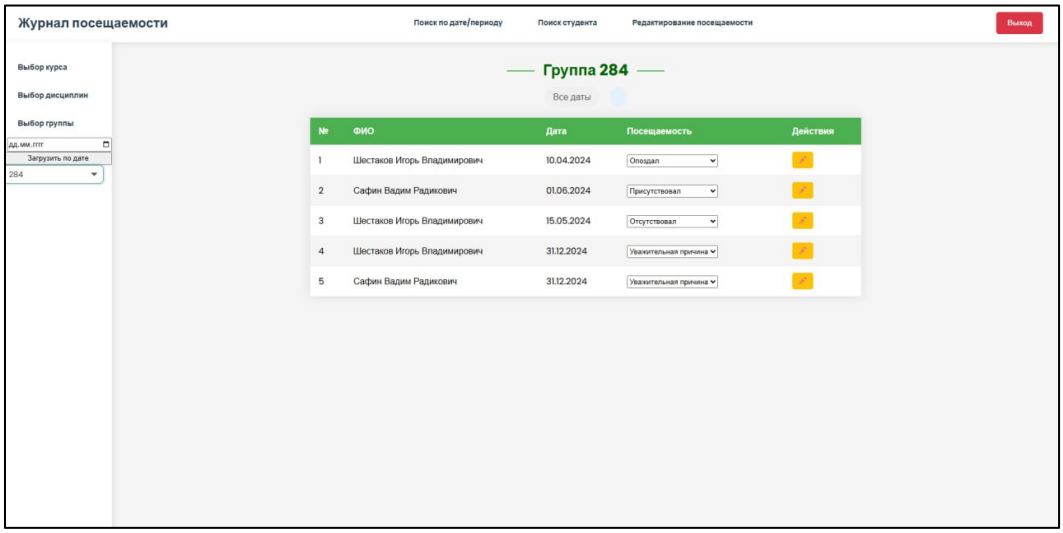

Рисунок 5.3 - Просмотр посещаемости выбранной группы

Следуя данной инструкции, пользователь может эффективно фиксировать посещаемость студентов по датам, просматривать историю конкретного студента или группы и при необходимости выполнять корректировки в соответствии с ролью в системе.

Заключение

Подводя итоги разработки, можно с уверенностью сказать, что поставленные цели и задачи выполнены в полном объёме. Созданное веб-приложение для управления продукцией мебельной компании корректно работает и готово к использованию в производственной деятельности или к дальнейшему расширению функциональности.

Внедрение данной системы позволит предприятию автоматизировать процессы учёта и редактирования данных о продукции, которые ранее выполнялись вручную. Это значительно ускорит работу сотрудников, снизит вероятность ошибок, связанных с человеческим фактором, и обеспечит актуальность информации о товарах в единой базе данных.

В ходе выполнения работы были решены следующие задачи: спроектирована и создана база данных в PostgreSQL, приведённая к третьей нормальной форме; разработано серверное приложение на Java с использованием Spring Boot, обеспечивающее обработку запросов и взаимодействие с базой данных через Spring Data JPA; создан пользовательский интерфейс с помощью шаблонизатора Thymeleaf, полностью соответствующий руководству по стилю; реализован функционал просмотра, добавления и редактирования продукции; разработан метод расчёта общего времени изготовления продукции на основе данных о цехах; создан метод расчёта необходимого количества сырья с учётом коэффициента типа продукции и процента потерь материала; реализована обработка ошибок с информированием пользователя через цветные уведомления.

Тема проекта оказалась многогранной и интересной, поскольку потребовала освоения и практического применения современных технологий: Spring Boot, Spring Data JPA, PostgreSQL, Thymeleaf. В процессе выполнения работы были углублены знания уже знакомых инструментов и изучены новые подходы к построению веб-приложений.

Таким образом, проект можно считать успешно выполненным. Полученное решение готово к практическому применению и может служить основой для дальнейшего развития информационной системы мебельного производства, например, добавления функционала управления заказами, партнёрами или складскими запасами.

Приложение А

Реализация метода расчёта количества сырья MaterialCalculator.java

```
package com.practice.uchebnayapraktika.Services;

import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import com.practice.uchebnayapraktika.Entitys.MaterialType;
import com.practice.uchebnayapraktika.Entitys.ProductType;
import com.practice.uchebnayapraktika.Repositories.
MaterialTypeRepository;
import com.practice.uchebnayapraktika.Repositories.
ProductTypeRepository;

@Component
public class MaterialCalculator {

    @Autowired
    private ProductTypeRepository productTypeRepository;

    @Autowired
    private MaterialTypeRepository materialTypeRepository;

    public int calculateRawMaterialQuantity(
            Integer productTypeId,
            Integer materialTypeId,
            Integer productQuantity,
            Double param1,
            Double param2) {

        // Проверка входных данных
        if (productTypeId == null || materialTypeId == null ||
            productQuantity == null || param1 == null || param2
== null) {
            return -1;
        }
        if (productQuantity <= 0 || param1 <= 0 || param2 <= 0) {
            return -1;
        }

        // Получаем тип продукции из БД
        Optional<ProductType> productTypeOpt =
productTypeRepository.findById(productTypeId);
        if (productTypeOpt.isEmpty()) {
            return -1; // несуществующий тип продукции
        }

        // Получаем тип материала из БД
```

```
        Optional<MaterialType> materialTypeOpt =
materialTypeRepository.findById(materialTypeId);
        if (materialTypeOpt.isEmpty()) {
            return -1; // несуществующий тип материала
        }

        ProductType productType = productTypeOpt.get();
        MaterialType materialType = materialTypeOpt.get();

        // Берем коэффициенты из БД (с защитой от null)
        double coefficient = productType.getCoefficient() != null
?
                                productType.getCoefficient().doubleV
alue() : 1.0;
        double lossPercent = materialType.getLossPercent() !=
null ?
                                materialType.getLossPercent().double
Value() : 0.0;

        // Расчет
        double materialPerUnit = param1 * param2 * coefficient;
        double totalMaterial = materialPerUnit * productQuantity;
        double materialWithLosses = totalMaterial * (1 +
lossPercent / 100);

        return (int) Math.ceil(materialWithLosses);
    }
}
```